

Investigating and Designing for Trust in AI-powered Code Generation Tools

Ruotong Wang*
ruotongw@cs.washington.edu
University of Washington
Seattle, Washington, USA

Denae Ford
denae@microsoft.com
Microsoft Research
Redmond, Washington, USA

Ruijia Cheng*
rcheng6@uw.edu
University of Washington
Seattle, Washington, USA

Thomas Zimmermann
tzimmer@microsoft.com
Microsoft Research
Redmond, Washington, USA

ABSTRACT

Trust is a crucial factor for the adoption and responsible usage of generative AI tools in complex tasks such as software engineering. However, we have a limited understanding of how software developers evaluate the trustworthiness of AI-powered code generation tools in real-world settings. To address this gap, we conducted Study 1, an interview study with 17 developers who use AI-powered code generation tools in professional or personal settings. We found that developers' trust is rooted in the AI tool's perceived ability, integrity, and benevolence, and is situational, varying according to the context of usage. Existing AI code generation tools lack the affordances for developers to efficiently and effectively evaluate the trustworthiness of AI-powered code generation tools. To explore designs that can augment the existing interface of AI-powered code generation tools, we explored three sets of design concepts (suggestion quality indicators, usage stats, and control mechanisms) that derived from Study 1 findings. In Study 2, a design probe study with 12 developers, we investigated the potential of these design concepts to help developers make effective trust judgments. We discuss the implication of our findings on the design of AI-powered code generation tools and future research on trust in AI.

CCS CONCEPTS

• **Human-centered computing** → **Empirical studies in HCI**; **HCI design and evaluation methods**; • **Software and its engineering**; • **Computing methodologies** → *Artificial intelligence*;

KEYWORDS

software engineering tooling, human-AI interaction, trust in AI, generative AI

ACM Reference Format:

Ruotong Wang, Ruijia Cheng, Denae Ford, and Thomas Zimmermann. 2024. Investigating and Designing for Trust in AI-powered Code Generation Tools.

*This work is done during the author's internship at Microsoft Research.



This work is licensed under a Creative Commons Attribution International 4.0 License.

FAccT '24, June 03–06, 2024, Rio de Janeiro, Brazil

© 2024 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-0450-5/24/06

<https://doi.org/10.1145/3630106.3658984>

In *The 2024 ACM Conference on Fairness, Accountability, and Transparency (FAccT '24)*, June 03–06, 2024, Rio de Janeiro, Brazil. ACM, New York, NY, USA, 19 pages. <https://doi.org/10.1145/3630106.3658984>

1 INTRODUCTION

With the rapid development of generative AI in recent years, it's increasingly used to support various human tasks in multiple domains, including complex information work such as software engineering. In software engineering, AI-powered code generation tools such as GitHub Copilot [3] and Tabnine [2] have quickly gained popularity in programmer communities [20, 35], enabling a new way of programming assistance [5, 50]. AI code generation tools can generate multiple lines of code in real-time based on a prompt within an Integrated Development Environment (IDE) [50].

While researchers and software developers are excited about AI-powered code generation tools, these tools also introduce new design challenges in creating responsible and reliable user experiences. One significant challenge involves helping users evaluate the trustworthiness of AI tools. Software developers' trust in programming support tools has long been studied as a crucial design requirement for such tools, as it serves as a key prerequisite for the safety of resulting software products [22, 26, 38]. Without proper support, developers can find it challenging to form accurate mental models of what AI tools can do or not [50] or determine the quality of specific AI suggestions [5, 48, 55]; thus becoming vulnerable to over- or under-trusting the AI [22, 46].

Existing research on trust in AI shows that the trustworthiness of technology is not inherent in an AI system but is based on how users interpret the information *communicated* via the systems' interfaces and interactions [36], and it can shift by context (e.g., task difficulty) [32, 70]. Yet, while emerging work has begun investigating the general usability of generative AI assistant tools in software engineering or broader domains [5, 6, 50, 55, 58, 71], we still know little about how their interfaces should be designed to *communicate* appropriate levels of trustworthiness and help developers form calibrated trust attitudes in AI-powered code generation tools.

In this paper, we present results from a two-stage qualitative study. We started by getting an empirical understanding of developers' notions of trust in the particular context of using AI code-generation tools. In Study 1, we conducted interviews with 17 developers who have various levels of experience in using AI-powered code generation tools in real-life scenarios. We analyzed

the results from Study 1 to answer the questions of *what factors contribute to developers' trust attitudes in AI-powered code generation tools* (RQ1) and *what challenges do developers face in evaluating the trustworthiness of AI tools* (RQ2). We found that developers evaluate the trustworthiness of an AI tool based on its perceived practical benefits, alignment with their short- and long-term goals, and process integrity when generating outputs. Moreover, developers continuously reassess these factors in specific contexts based on situational factors such as stakes or complexity of tasks, forming situational trust attitudes. We also found that the lack of trust affordances in existing AI tools could result in inefficient and biased evaluation of AI's trustworthiness. To explore solutions to these challenges, we explored *how to augment existing system interface to support effective and efficient evaluation of AI's trustworthiness* (RQ3) in Study 2. Specifically, we collected feedback on three groups of visual design concepts in design probe sessions with 12 additional developers. We found that design concepts, including quality indicators of AI suggestions, usage statistics dashboards, and control mechanisms to communicate user intention, show promise in scaffolding developers' trust judgments.

Our studies make the following contributions: (1) Building on prior literature that shows trust is rooted in the interplay between system characteristics and contexts [41] and the call for empirical understanding of trust in specific application areas [32], we provide a nuanced description of developers' notion of trust in generative AI tools in the context of programming, based on in-depth empirical data collected from interviews with developers who have real-world experience using AI powered code generation tools; (2) Furthering the growing literature on users' experiences with AI-powered code generation tools, we show the lack of ways to communicate users' intentions and lack of signals to validate AI output which are often characterized as usability challenges could, in fact, pose challenges for users to evaluate the trustworthiness of AI tools; (3) We contribute three groups of user-evaluated design implications, coupled with visual examples, to help designers take trust into consideration when designing AI-powered code generation tools.

2 RELATED WORK

2.1 Trust in AI

Trust is considered a key factor affecting user interaction with AI [16, 19, 36]. The lack of trust can prevent users from adopting AI tools in their workflow, even when the system's performance is superior [8, 47]. On the other hand, blind trust in AI, especially in high-stake tasks such as software engineering, can result in overlooking mistakes or risks produced by AI [48, 49].

Trust in AI is defined as the user's attitude that "an agent will help achieve an individual's goals in a situation characterized by uncertainty and vulnerability" [33, 36, 56], and therefore is particularly important when users engage in high-stake scenarios where the mistakes could have significant repercussions [30]. A review paper highlights that trust in AI is subjective and should be studied as an *attitude* [56], distinguishing from reliance or compliance, which are often studied as a *behavior* [54]. Indeed, Mayer et al. characterized trust as "an affective construct that can vary depending on the context and experiences of a person rather than simply being

a rational or an objective reality." in their seminal paper on organizational trust [41]. In the context of AI, empirical evidence has also shown that users' trust is affected by contextual factors such as institution investment, other users' endorsement, and riskiness of task [33, 61, 69].

Besides contextual factors, prior research identified that three system properties, including the system's ability, benevolence, and integrity, shape users' trust [32, 41]. More recently, Liao et al. highlighted the importance of *interface and interaction design* in mediating users' trust in AI [36]. Specifically, Liao et al. introduced the notion of *trust affordances*, which are visual cues in the interface that indicate the system's trustworthiness. Users make trust judgments based on these trust affordances. Therefore, user interfaces and interactions play an important role in *communicating* the internal trustworthy characteristics of AI to users. Following this understanding of trust, there has been a call for AI systems that can communicate an appropriate level of trustworthiness through their design, supporting users in building calibrated trust that aligns with the system's actual trustworthiness [13, 30, 66].

Many prior HCI works have empirically investigated the effectiveness of various interface augmentations to support users in evaluating and calibrating trust. One common approach is to explain AI predictions and decisions using confidence score [58, 69] or visual explanations [65], which could give users means and metrics to assess the performance of AI and make informed trust judgments. A related approach is to support the interpretability of model mechanisms [37, 43, 53], increasing the predictability [18, 21] of AI behavior. However, the effectiveness of these transparency features is not persistent across studies. For example, Agarwal et al. found that model confidence scores could mislead users' perception of the quality of model output [4]. Another approach is to provide users with ways to control AI behavior. For example, research has shown that allowing users to co-create music with AI-powered tools [39] or collaborate on writing tasks with AI models [34] can foster a sense of control and ownership, leading to higher trust in the system. Lastly, it has been shown that cognitive forcing functions such as delay showing AI's output could encourage users to engage with AI output analytically and reduce over-trust in AI [14].

Despite the plethora of research on trust in AI, most centers on deterministic AI tools for classification or prediction tasks [56]. These studies highlight the opportunity to support users in evaluating the trustworthiness of generative AI systems and forming calibrated trust attitudes using interfaces and interactions. However, how existing insights translate to generative AI tools, especially in software engineering contexts, remains an open question. Characteristics such as the richer and more complex input and output space [53] and more flexible roles in human-AI collaboration [25] distinguish generative AI tools from the deterministic AI tools that are widely studied, but also introduces new challenges and opportunities in designing for users' trust.

2.2 Generative AI in software engineering: AI-powered code generation tools

The recent development of generative AI models unleashes new possibilities for AI tools to support complex human tasks [7], including software engineering [5, 6, 59, 71]. Software engineering is

a type of complex and high-demanding information work that often involves high cognitive load and stress [23, 24, 27, 50], and therefore demands high-quality support. As a means to provide support for this complex work, commercial AI-powered code generation tools such as GitHub Copilot [3] and Tabnine [2] have emerged and become a novel service to expert and novice code creators alike. These tools provide AI services powered by large language models trained on code data [3, 52] and suggest code based on user prompts and project context [10]. For instance, powered by the OpenAI Codex model [1], Copilot is an extension in code editors that can generate code suggestions as ghost text at the user's cursor location. When using AI code generation tools, users can write comments in natural language and prompt the AI to generate code that they can accept, reject, make edits, and choose from various candidates. The AI can also complete users' in-progress code within a single line or by completing the function. Compared to traditional code completion tools based on defined rules and documentation, AI-powered code generation tools produce longer and more contextually relevant code snippets by synthesizing new code that might not exist in any code base [6]. As AI code generation tools introduce a brand new interaction paradigm between developers and AI [44, 64], early empirical investigations show that developers struggle to adapt to the new interaction pattern, often having an incomplete mental model of what Copilot can do or not [50] and finding it challenging to review and evaluate the quality of AI-generated code [5, 6, 49, 55]. These known usability challenges motivate us to understand how developers evaluate the trustworthiness of AI-powered code-generation tools.

While user trust has long been considered crucial in the design of traditional programming support tools, such as compilers and version control systems [62, 63] to ensure software safety [26, 38], developers' trust in AI code generation tools needs even more nuanced and careful consideration [6, 17, 48] due to the uncertainty introduced by generative AI. For example, since the mechanism of generative AI is more opaque and the outputs are more difficult to anticipate than traditional developer support tools, developers must establish an appropriate level of trust with these AI tools and be cautious about the potential risks [6]. In particular, Widder et al. conducted an ethnography study in 2021 with developers who use deterministic code generation tools and uncovered 16 factors that affect their trust in the tool [61]. Given that trust is deeply embedded in contextual factors, the factors identified in prior empirical studies might change for AI-powered code-generation tools as system properties and the social and organizational contexts of usage shift.

The pressing need to support developers in building and calibrating trust and the gap in previous literature motivated us to conduct retrospective interviews with developers who have experience using AI code generation tools in real-life scenarios (Study 1). In our research setting, developers face real consequences if AI produces undesirable outcomes, which could help us uncover the interplay of trust and contexts. Building on the interview findings and literature on the design of trust affordances, such as transparency and users' control, we further conducted a design probe study to understand the design space of trust affordances that can support developers in evaluating the trustworthiness of AI-powered code generation tools (Study 2).

3 STUDY 1: HOW DO DEVELOPERS EVALUATE THE TRUSTWORTHINESS OF AI TOOLS?

To understand what contributes to developers' trust attitudes in AI code generation tools (RQ1), as well as their challenges in making trust judgments (RQ2), we conducted retrospective interviews with 17 developers who use AI-powered code generation tools in real-life professional or personal settings.

3.1 Methods: Retrospective Interview Study

3.1.1 Study Procedure: collect critical incident + retrospective interview. To capture the interplay between trust and specific contexts of usage, we adopt a method of critical incident sampling and retrospective interviews. A similar approach has been applied to study patients' trust during medical visits [60, 67] and interpersonal trust in business negotiations [45]. A week before the scheduled interview, we contacted participants via instant message and asked them to prepare for the interview by collecting their significant moments when using AI-powered code generation tools during the following week—i.e., moments where they were either particularly satisfied, disappointed, or surprised. Participants were asked to share the descriptions and screenshots of those moments with us and were reminded regularly throughout the week. These records of significant moments helped participants recall the nuances of their experience in the interviews, allowing us to understand their trust in AI tools in realistic contexts of use. During the 60-minute retrospective interview sessions, we asked participants about their general experience with AI tools and then asked them to walk through the significant moments they collected during the prior week. We specifically probed for factors that affected their trust attitudes in AI. The interviews were conducted from July to August 2022, and the study procedures received approval from the Institution Review Board.

3.1.2 Participants. We recruited 17 software engineers with diverse programming and AI tools experience. Participants were recruited from different organizations of a large technology company through messages shared in group chats and emails distributed to developers chosen randomly from a directory. We stopped recruiting after hearing repeating themes in the interviews. Our final sample consists of 15 male and two female participants, aged between 18 and 54 years, with varying degrees of work experience and seniority. Participants had programming experience ranging from 2–25 years. They reported working on different areas of development (e.g., front-end, back-end, data science) and were involved in various types of development tasks (e.g., modifying existing features, writing new features, writing tests, refactoring). All participants had experience using Github Copilot, with various frequencies (9 daily, 3 weekly, 3 monthly, 2 recently started) and experience using it in professional and personal settings. Two also had experience with Tabnine. Detailed profiles of our participants are included in the Appendix (Table 1).

3.1.3 Data Analysis. All interviews were video and audio recorded and later transcribed. Our analysis of the interview data followed the procedure of inductive thematic analysis [9]. The first two authors took detailed field notes and frequently discussed the emerging themes with the research team during data collection. Based

on the field notes and discussions with the research team, the first author developed an initial codebook, applied it to the interview data, and noted places where codes could be merged or refined. The research team then collectively refined and grouped the codes via discussion, deriving a final code book, which is then re-applied to the data. The final codebook consists of 39 codes that focus on factors affecting developers' general trust in AI tools, their process of evaluating specific AI suggestions, and their challenges in building trust in AI tools. Example codes include *"trust varied by situations"*, *"initial expectation affects trust building"* or *"trial and error to build trust"*.

3.2 Factors that contribute to developers' trust attitudes in AI tools (RQ1)

Aligning with prior literature that indicates systems' *ability*, *integrity*, and *benevolence* as important factors that contribute to users' trust attitudes [36, 41], we observed that developers trust a given AI-powered code generation tool when they perceive practical benefits (*ability*), alignment with their goals (*benevolence*), and trustworthy processes (*integrity*). We also observed that *situational factors*, such as stakes of the use scenario and the complexity of the programming task, mediate developers' trust attitudes.

3.2.1 Ability: AI tools' practical benefits. The ability of an AI tool is defined as its competence or performance [33, 36, 41]. In the context of AI code generation tools, we observed that developers commonly assess the ability of an AI tool based on its **practical benefits to their work**, often related to time saved or lines of code contributed. Instead of expecting AI to provide perfect solutions, developers value the ease of building upon AI's outputs. Even when recognizing that AI's suggestions *"may not be able to compile or run correctly"*, P16 still trusts the tool: *"because I can always go back and modify it a little bit, tune it maybe, and get it to output what I want."* P10 values AI's utility in *"lay(ing) the foundation very well."* At the same time, P13 pointed out the potential for trust erosion if the AI's suggestion requires extra time to verify and correct: *"If Copilot ever slows down ... I would consider not using Copilot anymore."*

3.2.2 Benevolence: alignment of goals between AI and developer. Benevolence refers to the alignment of the AI tool's goals and users' goals [36, 41]. When it comes to AI code generation tools, benevolence is the perception that the AI tool is designed with developers' best interests in mind, supporting not just immediate task completion but also their long-term goals, such as learning and career growth. Trust arises when developers are convinced that the AI tool **respects their personal preferences, learning goals, and career aspirations**. However, we observed many instances of distrust due to the mismatch between what developers expect from the AI and the tool's actual behavior, leaving the impression of AI being aggressive and obtrusive. Regarding immediate task completion, P13 often found AI's suggestions to create unnecessary *"visual clutter"* on the screen when they already knew what they wanted to write. P7 felt that they had to *"fight with Copilot"* to let unwanted suggestions go away. Developers also worry that using AI tools would hinder their personal and career growth in the long run (i.e., limiting learning opportunities or eventually replacing their jobs). For example, after accepting high-quality suggestions from AI for a while, P8 started to worry about losing their *"programming*

muscles." As P8 said, Copilot *"want to sit in my seat...It started as a co-pilot, but now it's the pilot and I'm becoming the co-pilot."* P7 echoed the sentiment and worried that *"(AI tool is) robbing me from the opportunity to actually use my brain,"* preventing them from improving their own programming skills.

3.2.3 Integrity: the model mechanisms. Integrity is defined as whether the operational process of AI is appropriate to achieve users' goals (e.g., fair and secure when making decisions) [36, 42]. Developers trust AI tools when they are informed about and agree with **the model mechanism**. As P17 puts it: *"knowing how it works gives me more trust because I think it's just whether I agree with your approach or not."* Developers specifically highlighted the need to understand AI tools' security and privacy implications. P5's trust in Copilot increased after reading that *"it's bound by all these privacy laws."* Others noted a lack of relevant information for them to understand AI tools' process integrity. For example, P16 desired *"an end-to-end transparent diagram with what's exactly going on"* so that they could know *"exactly what's being tested to make sure that this code is appropriately copyrighted."*

3.2.4 Situational factors: the stake and complexity of tasks. Developers' trust in AI tools is not an object translation of the tool's ability, benevolence, and integrity but a dynamic assessment of the system characteristics together with additional situational factors such as **the stakes of the scenario** or **the complexity of tasks**. Developers are more reluctant to trust AI tools in high-stake and high-impact scenarios, such as on codebases that could *"impact millions of customers and millions of dollars potentially"* (P13). In those cases, they would only allow AI tools to play a *"suggestive"* role (P10) instead of generating code that would go into production. In another example, P3 shared that they would trust Copilot when writing *"proof-of-concept type of the projects,"* but not in *"actual production setting."* The complexity of tasks also affects trust. While P2 trusts AI tools for smaller mundane tasks that are *"standard and common"* and *"involves less logic to do,"* they don't expect AI tools to be useful for *"open-ended stuff."* Others also decide against using AI tools in situations with special requirements due to a lack of trust. For example, P6 does not trust Copilot to generate code that satisfies accessibility and responsiveness requirements. P2 does not use Copilot when they need to share the code with others because they don't trust it to generate code *"in the most explainable way that other people would understand."*

3.3 Challenges in evaluating the trustworthiness of AI tools (RQ2)

While AI tools' ability, benevolence, integrity and situational inform developers' trust attitudes, our findings show that the design of current AI-powered code generation tools fails to adequately support developers to evaluate these factors, leading to inefficiency and bias in trust attitudes. We outline three key challenges in this section.

3.3.1 Biased trust attitudes due to lack of reliable source of information on AI ability. Given that the performance of generative AI varies greatly depending on the specific context and task, making informed trust judgments requires developers to have a clear understanding of AI tools' ability in different situations. However, we notice a lack of reliable sources of information for developers

to understand the ability of AI tools. As a result, developers commonly rely on intuitions accumulated from first-hand experiences of evaluating AI outputs to determine AI tools' abilities in different situations. Developers like P13 form intuitions by observing AI performance in routine programming tasks: *"once you've seen it 10 times, I'm pretty sure Copilot will do this thing the 11th time."* Others like P6 *"played around"* or intentionally experiment with AI to try to *"break it"* when first started using Copilot, so that they can *"know where its limits are"*, which helped *"set my expectations on how to use it."*

However, the sole reliance on developers' personal experience can be inefficient. P13 shared that: *"you have to give it the benefit of the doubt for a while until it makes a little more sense to you as a tool...You just have to ignore those things until your expectation lines up with Copilot's capabilities."* It also leads to biased perceptions of the trustworthiness of AI tools. We observed that while positive experiences with AI tools lead to increased trust, negative experiences disproportionately impact developers' trust. A single misstep could instantly undermine their trust. For example, when P5 started using Copilot and found its multi-line suggestion to be unhelpful, they decided that they would *"not even read into the [multi-line] recommendations."* P5 further emphasized that: *"it takes three good recommendations to build trust versus one bad recommendation to lose trust."* The issue is exacerbated when developers bring expectations from traditional non-AI-based auto-completion tools such as IntelliSense, which pulls error-free code directly from the documentation. This creates unrealistic expectations for AI tools that generate more flexible suggestions that usually require reviews and edits and could lead to disappointment. P17 commented that *"It (AI tools) has to be better than IntelliSense to be worth using."* P14 also shared their frustration when observed that Copilot did not give them *"the right solution"* or, in fact, the same solution as IntelliSense.

3.3.2 Ineffective and inefficient evaluation of AI output. While evaluating each specific instance of AI suggestions forms the basis of developers' understanding of the AI tool's ability, we observed that developers often rely on inefficient manual methods due to inadequate support for evaluating the suggestion quality. The common strategies that developers use, such as *"logically going through the problem,"* (P11) or *"validat[ing] by testing it,"* (P13), can be time-consuming and ineffective. P9 shared that they spent half an hour identifying a small error of an additional bracket in a long block of code suggestions that spanned multiple lines. Some developers turn to external tools such as refactoring tools or library documentation for assistance. However, frequently using these methods could disrupt their programming workflow. The process of constantly switching between writing and reviewing code was described to be *"mentally draining"* (P7), *"derail my mind."* (P9) and eventually *"creates more work"* (P8). For example, P1 once had to spend extra effort researching a method they were not familiar with to debug: *"I had to look back at documentation, and it was using fields that were deprecated or nonsense fields that just created on its own."*

3.3.3 Lack of mechanisms to align AI with developers' goals and preferences. Aligned goals between developers and AI tools indicate the benevolence of the tool. However, the current interaction paradigms of AI code generation tools that mostly rely on including information in in-progress code for the AI to produce desired outputs make it challenging for developers to communicate their

short-term and long-term goals and preferences to AI tools, not to mention signaling the benevolence of the tool. P1 and P8 find it difficult to tailor their prompts to guide AI output without sacrificing their programming flow. As a result, they chose not to trust AI suggestions because *"there's no reason to expect Copilot will read my mind and figure out what I want to do now."* (P1) P9 desired a *"sensei"* version of Copilot that is more *"endearing"* and would *"invest in you by suggesting what you could learn"*, but find no way to communicate their goal. Another common challenge is signaling the desired timing of suggestions from AI tools. Many developers expressed frustration that they did not get enough suggestions when they desired AI's help; whereas other times they found AI suggestions to be intrusive, getting in the way of their programming flow. For example, P11 did not want *"Copilot to jump the gun and suggest before I finish fully defining the method"* because it would likely lead to *"suggestions that are way off the mark."*

3.4 Summary of results

Our findings in Study 1 reveal that developers tend to trust AI tools when they perceive practical benefits, alignment with their goals, and trustworthy processes. Furthermore, developers adjust their trust by considering additional situational factors such as task complexity and importance. However, the current AI tools do not provide enough support for the developers to assess AI tools' ability and benevolence in specific situations, resulting in inefficient and biased evaluation of the trustworthiness of AI tools. These findings motivate us to explore ways to improve the existing interface and interaction design of AI code generation tools to help developers more effectively calibrate their trust attitudes.

4 STUDY 2: HOW TO SUPPORT DEVELOPERS TO EVALUATE THE TRUSTWORTHINESS OF AI TOOLS?

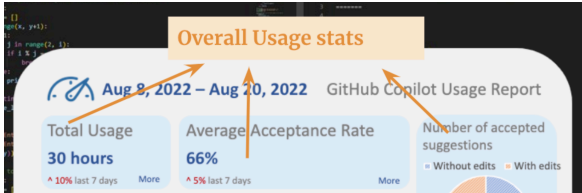
We conducted a design probe study to further explore how to augment existing system interfaces to support effective and efficient evaluation of AI's trustworthiness (RQ3). Building on findings in Study 1, we developed three groups of design concepts with visual representations and collected feedback from 12 developers. Notably, we do not aim to settle on or quantitatively evaluate the effectiveness of any specific design—rather, we used the designs as stimuli to elicit developers' feedback and aim to explore the potential of these interface design concepts as trust affordances through nuanced qualitative exploration. A similar approach has been used to explore interface designs for AI-assisted decision-making systems in child welfare [31] and clinical diagnosis [66].

4.1 Developing design concepts and visual stimuli

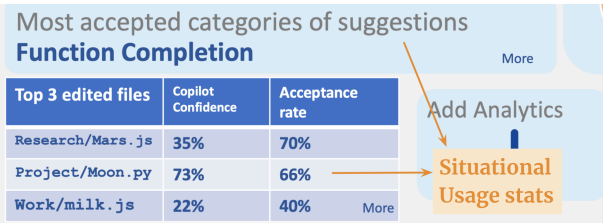
We first brainstormed design concepts that can address each of the challenges from Study 1. Some concepts were directly inspired by participants' interviews (e.g., control of timing, confidence score), while others were informed by literature (e.g., XAI features in [36], uncertainty visualizer in [53]). Once the team settled on the three groups of concepts, the first author created the initial visuals, which were then iterated with additional feedback from the team and pilot

participants. We intentionally kept the visuals low-fidelity since we wanted participants to focus on evaluating the *high-level concepts* of the design instead of the usability of specific graphical or textual elements, following the suggestion in [12]. All visual representations follow the design style of Copilot in Visual Studio Code since this combination was most commonly mentioned by participants in Study 1. In this subsection, we highlight the main features of each design concept and include the full visual representations in Appendix G).

4.1.1 Usage statistics dashboard to allow structured reflection on AI capability. Study 1 reveals that developers’ sole reliance on intuitions accumulated through personal experiences can lead to biased assessments of AI tool’s trustworthiness in different situations (§ 3.3.1). This points to a need for a more **structured approach to explicitly communicate AI tool’s strengths, limitations, and applicability in specific contexts** to help developers understand and reflect on their trust attitude. Specifically, we designed a dashboard that displays personalized usage statistics to developers, with comparisons with AI tools’ objective performance metrics in specific situations. The dashboard appears as a pop-up in the IDE after a user has used the AI tool for a certain period of time. The dashboard contains users’ overall usage stats (Figure 1a) and usage stats broken down by files (Figure 1b). The overall usage statistics include data such as total hours of usage and average acceptance rate, which help developers reflect on their interaction with the AI tool. The situational usage statistics include data such as the most accepted categories of suggestions, which enable developers to calibrate their trust according to different contexts. The comparisons between users’ acceptance rates and AI tools’ confidence in different contexts serve as a reality check against developers’ expectations. Users can access the dashboard via a button whenever they want to see it, allowing developers to dynamically recalibrate their trust based on ongoing usage.



(a) Overall usage stats



(b) Situational usage stats

Figure 1: A usage statistics dashboard that displays personalized usage statistics to a user. Both (a) overall usage stats and (b) situational usage stats are shown in a pop-up dashboard in IDE.

4.1.2 Quality indicators to support efficient in-context evaluation of AI suggestions. Evaluating each instance of AI suggestions helps developers build up their understanding of AI’s abilities and enables developers to integrate AI output into their workflow (§ 3.2.1). However, the lack of support for the evaluation process forced developers to rely on manual methods or external tools (i.e., documentation), which are often time-consuming, ineffective, and disrupt developers’ workflow (§ 3.3.2). Thus, we created design concepts to **provide in-context support that enhances the evaluation process without disrupting the workflow**. Concretely, we explored three ways to provide transparency into the AI model’s confidence in the output as non-disruptive ways to help developers make quick and accurate assessments of the quality of suggestions. The *Solution-level confidence explanation* (Figure 2a) indicates the model’s aggregated confidence of the solution in the editing window, helping developers to quickly decide whether to build upon the AI’s suggestion or discard it. If developers decide to scrutinize the suggestion closely, the *Token level confidence/uncertainty explanation* (Figure 2a) highlights specific tokens in the solution where the model has low confidence, helping developers to identify potential problems in the suggestion. Finally, the *File-level familiarity explanation* (Figure 2b) communicates the model’s familiarity and alignment with the specific context in the file. For example, if the model has not seen input in the specific programming language or is using a particular library, the familiarity indicator might turn yellow or red to indicate that the model is unfamiliar with the context provided in the file.

4.1.3 Control mechanisms at the onset and during programming session to help align developers and AI’s goals. Existing AI code generation tools require developers to include information in the code they are working on to produce desired AI outputs. However, this interaction paradigm makes it challenging for developers to communicate their intentions and thus evaluate AI tool’s benevolence (§ 3.3.3). Therefore, it is crucial to provide developers effective ways to **convey short-term and long-term goals and preferences**. To help bridge this communication gap, we designed two mechanisms for developers to indicate intention (§ 3.2.2) and preferences for AI’s approach (§ 3.2.3) when generating suggestions. To complement the existing natural language interface, we designed control mechanisms in graphical interfaces. Specifically, we designed a control panel (Figure 3a) that enables developers to set explicit intentions and define goals for using the AI tool at the project initialization. In the control panel, developers can specify specific benefits they expect to gain from using the AI tool in the programming sessions (e.g., to help them speed up by serving as a prototyping tool or to help them learn as a programming tutor). We chose to use system roles as metaphors since it has been shown to effectively bridge the communication gap between users and large language models [51]. Users can also further customize settings by adjusting the configuration on the right side of the control panel. We included options such as suggestion scope, the maximum length of suggestion, the timing of suggestion, and the type of validation (e.g., only suggest solutions that pass security checks). We also designed a context adjustment slider (Figure 3b) that enables developers to adapt AI behavior further during the programming sessions. Users can drag

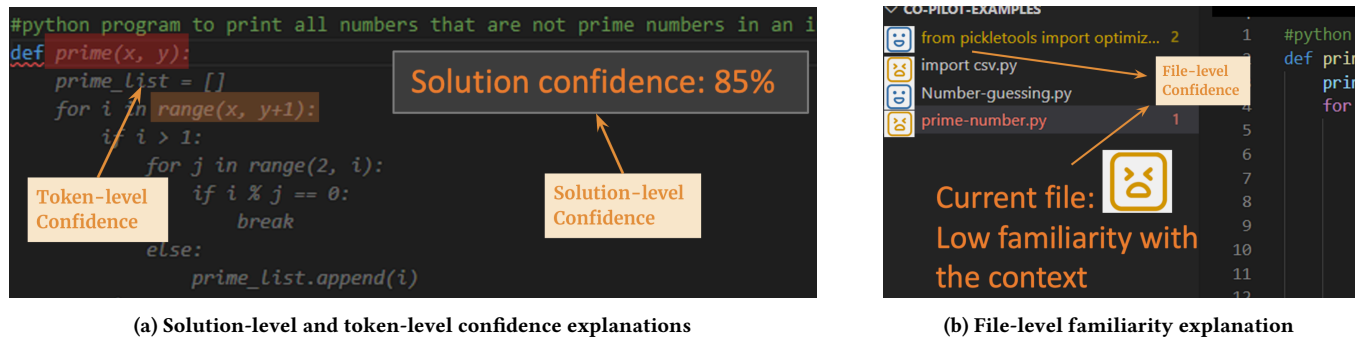


Figure 2: Quality indicators to support users better evaluate each AI suggestion.

the control bar next to each file name or the code snippets to manually select the context they would like to include as part of the prompts for code generation.

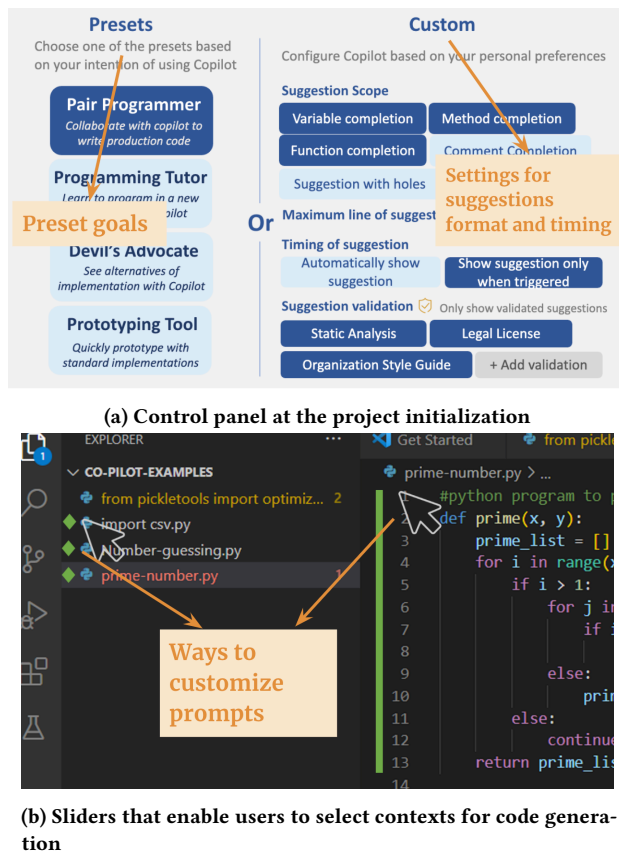


Figure 3: Two control mechanisms that allow users to communicate intentions to the AI tool. (a) control panel allows users to select system roles at the project initialization; (b) allows users to adapt AI behavior during the programming sessions.

4.2 Study procedure, participants, and data analysis

We conducted one-to-one 60-minute design probe sessions with 12 developers with diverse programming experience and experience with AI code generation tools from social media and a large technology company. To recruit participants, we emailed 600 randomly selected developers and advertised on social media. We selected participants with various levels of experience with AI tools while ensuring diversity in race, age, and work experience. We stopped recruiting after hearing repeating themes in the interviews. Our final sample includes nine males and three females from different racial groups whose programming experience ranges from 4 to 45 years. All participants in Study 2 have experience with Copilot - 8 use it regularly, 2 recently started using it, and two have used it but are no longer using it. Detailed profiles of our participants are presented in the Appendix (Table 2). To capture a broader range of experiences, we didn't invite participants in Study 1 to participate in Study 2 again.

The co-design session starts with brief questions on developers' trust attitudes toward AI code generation tools. We then showed the three sets of design concepts to the participants. The visual representations of the design concepts were presented in Microsoft PowerPoint. Each concept is animated to show a sequence of actions to demonstrate the interaction. During the session, we explained each design and asked for participant feedback and reactions, including questions on how they imagined using the proposed features in real life and if and how the features contribute to trust. We also encouraged participants to brainstorm new features. The study procedures received approval from the Institution Review Board.

Similar to Study 1, all sessions were video and audio recorded and later transcribed. The data analysis followed the procedure of deductive thematic analysis [9], following the structure of each design concept. In the analysis, we focused on analyzing ways that interface features are helpful or not helpful for participants to evaluate the trustworthiness of the AI tool, especially the factors identified in Study 1. We also looked for potential risks and places of improvement for each design concept.

4.3 Study 2 findings

4.3.1 Demonstrating AI's practical benefits via usage statistics. Participants found that the explicit information on Copilot's abilities in

both overall and situational usage dashboards was helpful for aligning their expectations with AI's ability. For example, P8 thought that *"the aggregated measures of how I've used Copilot over time helps me form an image of my relationship with Copilot, which helps me evaluate Copilot's performance and form informed goals."* Several participants (P5, P6, P10) agree that statistics such as suggestion acceptance rate and time saved are useful for demonstrating the AI tool's practical benefit and can help them calibrate their trust in it. For example, P10 suggests: *"if I can see a quantifiable number of how much Copilot increases my productivity or saved me time, I'm more prone to depend on it more."* While we included file-level statistics to help developers calibrate their trust for different situations, P8 wished to see more granular breakdowns of Copilot's performance based on functional concepts so that they can better *"navigate that space with which topics is Copilot the best at."*

At the same time, it can be challenging for users to interpret the statistics shown on the dashboard. For example, P9 worried that: *"I also need to analyze the correlation and causation, the statistical numbers. I think it's just put into many works to developers."* In addition to the numbers, P7 prefers more actionable insights: *"it's the performance of the Copilot, not my performance, so there's nothing that I can change just based on this...to improve working efficiency with the Copilot."* Similarly, P2 wished that the dashboard could not only tell users *"how users used things,"* but also *"how to use something,"* by including some actionable tips on how to use Copilot in unobtrusive ways. In addition, participants were concerned about potential privacy issues, especially for workplace surveillance when tracking telemetry data. For example, P12 worried that organizations would use the tracked data to evaluate employees.

4.3.2 Offering quality indicators to support evaluation of AI suggestions. Participants found that the quality indicators at different levels are helpful for them in more efficiently and effectively assessing the quality of code suggestions. For example, P2 thought that the file-level confidence indicates the helpfulness of the AI tool in nuanced and accurate ways: *"If I know the Copilot is not very familiar with this code, I am not going to have high expectations that the code the Copilot produces will be accurate."* P8 thinks that additional transparency helps them make quick and reliable trust judgments: *"low familiarity can be a sign of vulnerability for the machine. If I know [the AI tool] is not good at it. I will be more vigilant, careful when I'm writing the code myself or incorporating it... [the transparency signals] help me know how much I should be relying on it."* These signals also help prompt subsequent user actions, helping developers integrate AI suggestions into their workflow. P5 uses the highlights of low-confidence tokens to guide their validation process and *"target where I'm reviewing the logic and say, yeah, it wasn't super confident about these parts, so I should look more closely at what it did there."*

At the same time, developers indicated that the transparency signals could be hard to interpret without additional contexts. For example, P5 thought that the solution-level confidence indicators were not very helpful because: *"Even that 20 percent, maybe I have to tweak five lines, it's still a win."* P7 also expressed a similar reluctance to fully rely on the numeric metrics: *"I will pick that a solution even though the confidence score is a little bit lower than the others because it meets my needs better. Human judgment actually*

knows that that is a better solution." Indeed, many developers also reported challenges in interpreting the context of model confidence numbers—the same numeric score could communicate different information for different developers in a variety of scenarios. Lastly, explicit indications of model confidence also introduced potential bias in users' trust judgments, as users may be *"more likely to accept without critically thinking about a suggestion"* or *"reject a valid solution or a valid suggestion based on low familiarity, even though it's a perfectly valid solution that is ultimately productive."* (P6)

4.3.3 Communicating developers' intention with control mechanisms. Participants found the control panel at project initialization and the context adjustment sliders during programming sessions helpful for aligning AI tools with their specific intentions and preferences. The context adjustment sliders offer *"more tools to guide Copilot to the right answer"* and allow developers to: *"teach the model what to do for me when I need it."* (P1) The control panel, on the other hand, allows developers to customize how much and what kind of help they get from Copilot at project initialization, which makes the AI more predictable and controllable. For example, P10 once worried that Copilot might introduce unnoticed security bugs, but the option in the control panel for users to customize the type of suggestions could allow them to *"only get suggestions that have been scanned for any security vulnerabilities."* Indeed, control mechanisms allowed users to customize a more reliable and helpful version of Copilot, as P7 described, *"if the performance is not reliable anymore or if there are suggestions that I don't need, I would turn those off those function just for precision and clarity."* Trust was fostered in the process since users felt they had control over what and how the AI will make suggestions: *"the ability to set a boundary [for Copilot] and have it respect that boundary is the core of building trust. If it can work in that boundary, then you trust it more, and you can give it more permission."* (P4)

Interestingly, although we did not explicitly design the control mechanisms to inform users' expectations of AI's abilities, developers thought the control mechanisms allowed them to develop more concrete expectations of what AI can and cannot do. For example, P5 thought that seeing all possibilities to control is almost like interactive documentation for the AI tools' functionalities, showing the full capacity of AI tools. P12 thought that the control panel is especially helpful in project initialization because it allows them to have *"concrete expectations of what is going to happen,"* such as *"how many lines of code there will be in suggestions."* Others imagined experimenting with functionalities using the controls to understand the strengths and limitations of Copilot in more targeted ways. For example, P7 imagined themselves to *"turn off everything and see what each function does and see which functions are more helpful,"* which allowed them to have *"the full scope of what Copilot does."*

At the same time, developers expressed concern that too much control could be a burden for users. For example, P5 expressed doubts about the usefulness of the context slider due to its high interaction cost: *"if I start spending a bunch of time managing what context it has that the utility starts dropping because I'm investing more time than am I getting anything more out of it."* The choice of what type of controls to grant users and how to foreshadow their impact on AI behaviors also needs careful consideration. A few developers were confused about some of the current designs, and

hoped to see more examples in action and “*visual cues for what this looks in the editor*” (P5) or “*examples like how the code will be different, like turning it on and off*” (P7), on top of the textual description of the control mechanisms. P10 also thought the presets could be helpful, “*especially for someone who has no idea about all these customization settings*.”

5 DISCUSSION

5.1 Trust in generative AI tools

Building on prior work that calls for real-world empirical studies of users’ trust in AI tools [32], our work contributes a detailed account of users’ notions of trust in AI code generation tools based on retrospective interviews with developers who have used such tools in real-life scenarios. Aligning with theories in existing literature [36, 41], we observed that developers’ trust attitude in AI-powered code generation tool is informed by the tool’s perceived practical benefits (*ability*), alignment with developers’ goals (*benevolence*), trustworthy processes (*integrity*) and *situational factors*, such as stakes of the use scenario and the complexity of the programming task. This echoes prior work indicating that trust is evolving over time [29], is situational [28, 30, 33, 70] and affected by social and organizational contexts [32, 61].

Responding to recent calls to understand how cues in the design of system interface (i.e., trust affordance) communicate the internal trustworthy characteristics of AI to users [36], we observed a lack of trust affordances that can effectively convey the trustworthiness of AI-powered code generation tools. As a result, developers are forced to rely on intuitions accumulated from their limited personal experiences to make trust judgments, which can be inefficient and ineffective and lead to biased trust attitudes. Although our data focused on developers’ challenges with AI code generation tools, the challenges of evaluating AI output [71] and conveying goals and intentions to AI using natural language are also observed in other applications of large language models [25, 40, 68]. Our work highlights that these challenges not only manifest as usability problems but also affect users’ judgment of the trustworthiness of generative AI applications, leading to a potential overreliance on AI or preventing users from taking full advantage of AI. Our work also shows that graphical user interface (GUI) remains crucial in assisting users in establishing *calibrated* and warranted trust in AI, despite recent debates on the possibility of replacing the conventional GUI with the emerging language user interface (e.g., [57]).

5.2 Design for trust affordances in AI code generation tools

Findings from our design probe study (study 2) additionally shed light on opportunities to support users in building and adjusting their trust in AI tools by augmenting existing interfaces with trust affordances. We outline specific design implications below. While the specific recommendations are derived from the context of AI code generation tools, we believe our advice can also be useful for supporting users in building and calibrating trust with generative AI applications more broadly.

5.2.1 Encourage structured reflection on AI tool’s performance and applicability in specific contexts. Developers’ trust attitudes are

often informed by intuitions accumulated from their personal experiences with AI tools, which can lead to bias and inefficiencies in calibrating their trust in different situations. This suggests a more **structured approach to align users’ expectations by explicitly communicating AI tools’ performance and applicability in specific contexts, while also encouraging users’ to reflect on the gap between their perception and the tool’s actual performance.** In study 2, we evaluated a feedback analytic dashboard that shows personalized statistics of AI tools’ performance in different contexts (Figure 1a and 1b), which proved to be effective in helping developers to form accurate expectations and understand the tool’s utility. However, we noticed that simply showing comparisons of statistics might not be enough to prompt users to engage in a reflection, as they can be hard to interpret and require certain data literacy. Therefore, further systems could consider providing more explicit guidance on how users should adjust their trust attitudes or including actionable suggestions on how users can effectively engage with AI tools (e.g., tips on when to use the tool).

5.2.2 Support evaluation of AI output using context-aware quality indicators. Findings from study 1 show that while evaluating AI output forms the basis of trust attitude, developers rely on native methods such as eye-browsing or running the program, which is time-consuming and ineffective, calling for the need to provide **in-context support for developers to make quick and accurate evaluations of AI output.** In study 2, we explored the potential of three levels of model confidence scores of AI suggestions: token level, solution level, and file level (Figure 2a and 2b). While developers find the confidence indicators useful for evaluating solutions and guiding their actions, there’s a clear need to customize these quality indicators to suit diverse preferences and requirements (e.g., explainability or accessibility requirements). One possible design is to allow users to define or adjust metrics based on their specific needs and contexts. The quality indicators could also go beyond explanations of modal mechanisms to include social transparency, such as acceptability of the solution in the community [15]. Lastly, as previous research on confidence scores has suggested, the design should be wary of users’ overreliance [4]. To mitigate this, it’s vital to present quality indicators as part of a broader evaluation framework that includes clear explanations of their meaning and appropriate use. For instance, rather than solely relying on numerical confidence scores, which can be misleading or hard to interpret, AI tools could explain why certain parts of the code were flagged as low confidence to encourage critical reasoning.

5.2.3 Afford users to convey short- and long-term goals and preferences. The various ways that AI tools can be used make it important to help users communicate their intentions clearly. In the design probe study, we demonstrate examples of control options that allow users to customize the timing, characteristics, as well as local context of AI suggestions (Figure 3a). These means of control allow AI tools to better align with users’ goals and intentions, communicating the benevolence of the system. This also echoes prior research in the context of AI-powered music generation which indicated that enabling users to steer AI behavior increases trust in AI [39]. However, more controls come with more responsibilities. Designers of generative AI systems need to be cautious about overburdening users with decisions that they are not confident in making or less

important to their experience. We suggest that control mechanisms should prioritize places where users have discrepancies or group options and provide users with the option to have simple defaults. We explored persona as a grouping mechanism, which proved helpful. Further systems could also imagine other ways to group them, such as stake of tasks or expertise of users. It's also important to consider how to explain and help users preview the outcome of different control options. Although the users reacted positively to the design probes, they also pointed to the challenges of understanding the control options. Future systems can explore how to introduce the control options more clearly. For example, an interactive onboarding session could potentially address the issue by demonstrating to users the effect of control options in action. Toolsmiths can even consider rolling out an incremental, progressive clarity on what control means.

5.3 Limitations and future work

In this study, we investigated developers' trust in AI-powered code generation tools via qualitative interviews. Future research can build on our qualitative investigation by implementing and evaluating interactive prototypes in controlled experiments to better quantify the effects of interface design on users' trust.

In addition, although we try to reach a diverse population in terms of demographic factors, our sample is still heavily skewed toward male developers, given the general demographics of software engineering. The skewed gender distribution might have affected our findings, given prior research showing that women and minority groups might have different preferences in programming activities (e.g., [11]). We call for future work to gain a more in-depth understanding of how female and gender minority developers approach trust in AI tools.

Further, our data in Study 1 were collected at a single company. Although we encouraged participants to also discuss their experience outside of the work in Study 1 and intentionally sampled outside of the organization in Study 2, there may be additional needs that we miss because of the specific organizational setting.

Lastly, we collected our interview data between July and August 2022, at a time when AI-powered code generation tools were just starting to emerge. Since then, the landscape of AI-powered code generation tools has been rapidly changing, with several new tools emerging. Existing tools such as GitHub Copilot also introduced updates such as conversation assistants and content exclusion settings. To better contextualize our findings, we provide a description of the features of GitHub Copilot and Tabnine as of July 2020 in the Appendix. Although the core interaction paradigm of AI suggesting code snippets based on code context and natural language prompts remains unchanged, we encourage future research to explore the effect on trust given the fast-growing adoptions in different communities and organizational settings [35].

ACKNOWLEDGMENTS

We would like to thank the participants for their valuable insights and anonymous reviewers for their helpful feedback. We would also like to thank members of the Microsoft Research SAINTES team and members of the University of Washington Social Futures Lab for their thoughtful discussion and feedback.

REFERENCES

- [1] 2021. OpenAI Codex. <https://openai.com/blog/openai-codex/>
- [2] 2023. AI Assistant for software developers | Tabnine. <https://www.tabnine.com/>
- [3] 2023. GitHub Copilot · Your AI pair programmer. <https://github.com/features/copilot>
- [4] Mayank Agarwal, Kartik Talamadupula, Stephanie Houde, Fernando Martinez, Michael Muller, John Richards, Steven Ross, and Justin D. Weisz. 2021. Quality Estimation & Interpretability for Code Translation. <https://doi.org/10.48550/arXiv.2012.07581> arXiv:2012.07581 [cs].
- [5] Shraddha Barke, Michael B. James, and Nadia Polikarpova. 2022. Grounded Copilot: How Programmers Interact with Code-Generating Models. <https://doi.org/10.48550/arXiv.2206.15000> arXiv:2206.15000 [cs].
- [6] Christian Bird, Denae Ford, Thomas Zimmermann, Nicole Forsgren, Eirini Kalliamvakou, Travis Lowdermilk, and Idan Gazit. 2023. Taking Flight with Copilot: Early insights and opportunities of AI-powered pair-programming tools. *Queue* 20, 6 (Jan. 2023), Pages 10:35–Pages 10:57. <https://doi.org/10.1145/3582083>
- [7] Rishi Bommasani, Drew A. Hudson, Ehsan Adeli, Russ Altman, Simran Arora, Sydney von Arx, Michael S. Bernstein, Jeannette Bohg, Antoine Bosselut, Emma Brunskill, Erik Brynjolfsson, Shyamal Buch, Dallas Card, Rodrigo Castellon, Niladri Chatterji, Annie Chen, Kathleen Creel, Jared Quincy Davis, Dora Demszky, Chris Donahue, Moussa Doumbouya, Esin Durmus, Stefano Ermon, John Etchemendy, Kavin Ethayarajh, Li Fei-Fei, Chelsea Finn, Trevor Gale, Lauren Gillespie, Karan Goel, Noah Goodman, Shelby Grossman, Neel Guha, Tatsunori Hashimoto, Peter Henderson, John Hewitt, Daniel E. Ho, Jenny Hong, Kyle Hsu, Jing Huang, Thomas Icard, Saahil Jain, Dan Jurafsky, Pratyusha Kalluri, Siddharth Karamcheti, Geoff Keeling, Fereshte Khani, Omar Khattab, Pang Wei Koh, Mark Krass, Ranjay Krishna, Rohith Kudithipudi, Ananya Kumar, Faisal Ladhak, Mina Lee, Tony Lee, Jure Leskovec, Isabelle Levent, Xiang Lisa Li, Xuechen Li, Tengyu Ma, Ali Malik, Christopher D. Manning, Suvir Mirchandani, Eric Mitchell, Zanele Munyikwa, Suraj Nair, Avaniika Narayan, Deepak Narayanan, Ben Newman, Allen Nie, Juan Carlos Niebles, Hamed Nilforoshan, Julian Nyarko, Giray Ogut, Laurel Orr, Isabel Papadimitriou, Joon Sung Park, Chris Piech, Eva Portelance, Christopher Potts, Aditi Raghunathan, Rob Reich, Hongyu Ren, Frieda Rong, Yusuf Roohani, Camilo Ruiz, Jack Ryan, Christopher Ré, Dorsa Sadigh, Shiori Sagawa, Keshav Santhanam, Andy Shih, Krishnan Srinivasan, Alex Tamkin, Rohan Taori, Armin W. Thomas, Florian Tramèr, Rose E. Wang, William Wang, Bohan Wu, Jiahui Wu, Yuhuai Wu, Sang Michael Xie, Michihiro Yasunaga, Jiaxuan You, Matei Zaharia, Michael Zhang, Tianyi Zhang, Xikun Zhang, Yuhui Zhang, Lucia Zheng, Kaitlyn Zhou, and Percy Liang. 2021. On the Opportunities and Risks of Foundation Models. <https://arxiv.org/abs/2108.07258v3>
- [8] Jayson G. Boubin, Christina F. Rusnock, and Jason M. Bindewald. 2017. Quantifying Compliance and Reliance Trust Behaviors to Influence Trust in Human-Automation Teams. *Proceedings of the Human Factors and Ergonomics Society Annual Meeting* 61, 1 (Sept. 2017), 750–754. <https://doi.org/10.1177/1541931213601672> Publisher: SAGE Publications Inc.
- [9] Virginia Braun and Victoria Clarke. 2019. Reflecting on reflexive thematic analysis. *Qualitative Research in Sport, Exercise and Health* 11, 4 (Aug. 2019), 589–597. <https://doi.org/10.1080/2159676X.2019.1628806> Publisher: Routledge _eprint: <https://doi.org/10.1080/2159676X.2019.1628806>
- [10] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. *Advances in neural information processing systems* 33 (2020), 1877–1901.
- [11] Margaret Burnett, Scott D Fleming, Shamsi Iqbal, Gina Venolia, Vidya Rajaram, Umer Farooq, Valentina Grigoreanu, and Mary Czerwinski. 2010. Gender differences and programming environments: across programming populations. In *Proceedings of the 2010 ACM-IEEE international symposium on empirical software engineering and measurement*. 1–10.
- [12] Bill Buxton. 2007. *Sketching User Experiences: Getting the Design Right and the Right Design*. Morgan Kaufmann Publishers Inc., San Francisco, CA, USA.
- [13] Zana Bućinca, Phoebe Lin, Krzysztof Z. Gajos, and Elena L. Glassman. 2020. Proxy tasks and subjective measures can be misleading in evaluating explainable AI systems. In *Proceedings of the 25th International Conference on Intelligent User Interfaces (IUI '20)*. Association for Computing Machinery, New York, NY, USA, 454–464. <https://doi.org/10.1145/3377325.3377498>
- [14] Zana Bućinca, Maja Barbara Malaya, and Krzysztof Z. Gajos. 2021. To Trust or to Think: Cognitive Forcing Functions Can Reduce Overreliance on AI in AI-assisted Decision-making. *Proceedings of the ACM on Human-Computer Interaction* 5, CSCW1 (April 2021), 188:1–188:21. <https://doi.org/10.1145/3449287>
- [15] Ruijia Cheng, Ruotong Wang, Thomas Zimmermann, and Denae Ford. 2024. "It would work for me too": How Online Communities Shape Software Developers' Trust in AI-Powered Code Generation Tools. *ACM Trans. Interact. Intell. Syst.* (mar 2024). <https://doi.org/10.1145/3651990> Just Accepted.
- [16] European Commission. 2019. Building Trust in Human-Centric Artificial Intelligence. Retrieved September 1, 2022 from <https://digital-strategy.ec.europa.eu/en/library/communication-building-trust-human-centric-artificial-intelli>

- [17] Arghavan Moradi Dakhel, Vahid Majdinasab, Amin Nikanjam, Foutse Khomh, Michel C. Desmarais, Zhen Ming, and Jiang. 2022. GitHub Copilot AI pair programmer: Asset or Liability? <https://doi.org/10.48550/ARXIV.2206.15331>
- [18] Sylvain Daronnat, Leif Azzopardi, Martin Halvey, and Mateusz Dubiel. 2021. Inferring Trust From Users' Behaviours; Agents' Predictability Positively Affects Trust, Task Performance and Cognitive Load in Human-Agent Real-Time Collaboration. *Frontiers in Robotics and AI* 8 (2021), 194. <https://doi.org/10.3389/frobt.2021.642201>
- [19] Arun Das and Paul Rad. 2020. Opportunities and Challenges in Explainable Artificial Intelligence (XAI): A Survey. <https://doi.org/10.48550/arXiv.2006.11371> arXiv:2006.11371 [cs].
- [20] Thomas Dohmke. 2022. GitHub Copilot is generally available to all developers. <https://github.blog/2022-06-21-github-copilot-is-generally-available-to-all-developers/>
- [21] Jaimie Drozdal, Justin Weisz, Dakuo Wang, Gaurav Dass, Bingsheng Yao, Changrui Zhao, Michael Muller, Lin Ju, and Hui Su. 2020. Trust in AutoML: exploring information needs for establishing trust in automated machine learning systems. In *Proceedings of the 25th International Conference on Intelligent User Interfaces (IUI '20)*. Association for Computing Machinery, New York, NY, USA, 297–307. <https://doi.org/10.1145/3377325.3377501>
- [22] Zakir Durumeric, Frank Li, James Kasten, Johanna Amann, Jethro Beekman, Mathias Payer, Nicolas Weaver, David Adrian, Vern Paxson, Michael Bailey, and J. Alex Halderman. 2014. The Matter of Heartbleed. In *Proceedings of the 2014 Conference on Internet Measurement Conference (Vancouver, BC, Canada) (IMC '14)*. Association for Computing Machinery, New York, NY, USA, 475–488. <https://doi.org/10.1145/2663716.2663755>
- [23] Neil A. Ernst and Gabriele Bavota. 2022. AI-Driven Development Is Here: Should You Worry? *IEEE Software* 39, 2 (2022), 106–110. <https://doi.org/10.1109/MS.2021.3133805>
- [24] Lucian Gonçalves, Kleinner Farias, Bruno da Silva, and Jonathan Fessler. 2019. Measuring the Cognitive Load of Software Developers: A Systematic Mapping Study. In *2019 IEEE/ACM 27th International Conference on Program Comprehension (ICPC)*. 42–52. <https://doi.org/10.1109/ICPC.2019.00018>
- [25] Matthew Guzdial, Nicholas Liao, Jonathan Chen, Shao-Yu Chen, Shukan Shah, Vishwa Shah, Joshua Reno, Gillian Smith, and Mark O. Riedl. 2019. Friend, Collaborator, Student, Manager: How Design of an AI-Driven Game Level Editor Affects Creators. In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems*. ACM, Glasgow Scotland Uk, 1–13. <https://doi.org/10.1145/3290605.3300854>
- [26] W. Hasselbring and R. Reussner. 2006. Toward trustworthy software systems. *Computer* 39, 4 (2006), 91–92. <https://doi.org/10.1109/MC.2006.142>
- [27] Daniel Helgeson, Emelie Engström, Per Runeson, and Elizabeth Bjarnason. 2019. Cognitive Load Drivers in Large Scale Software Development. In *2019 IEEE/ACM 12th International Workshop on Cooperative and Human Aspects of Software Engineering (CHASE)*. 91–94. <https://doi.org/10.1109/CHASE.2019.00030>
- [28] Robert R. Hoffman. 2017. A Taxonomy of Emergent Trusting in the Human–Machine Relationship. In *Cognitive Systems Engineering*. CRC Press. Num Pages: 28.
- [29] Daniel Holliday, Stephanie Wilson, and Simone Stumpf. 2016. User Trust in Intelligent Systems: A Journey Over Time. In *Proceedings of the 21st International Conference on Intelligent User Interfaces (IUI '16)*. Association for Computing Machinery, New York, NY, USA, 164–168. <https://doi.org/10.1145/2856767.2856811>
- [30] Alon Jacovi, Ana Marasović, Tim Miller, and Yoav Goldberg. 2021. Formalizing Trust in Artificial Intelligence: Prerequisites, Causes and Goals of Human Trust in AI. In *Proceedings of the 2021 ACM Conference on Fairness, Accountability, and Transparency (FAccT '21)*. Association for Computing Machinery, New York, NY, USA, 624–635. <https://doi.org/10.1145/3442188.3445923>
- [31] Anna Kawakami, Venkatesh Sivaraman, Logan Stapleton, Hao-Fei Cheng, Adam Perer, Zhiwei Steven Wu, Haiyi Zhu, and Kenneth Holstein. 2022. Why Do I Care What's Similar: Probing Challenges in AI-Assisted Child Welfare Decision-Making through Worker-AI Interface Design Concepts. In *Designing Interactive Systems Conference (DIS '22)*. Association for Computing Machinery, New York, NY, USA, 454–470. <https://doi.org/10.1145/3532106.3533556>
- [32] Sunnie S. Y. Kim, Elizabeth Anne Watkins, Olga Russakovsky, Ruth Fong, and Andrés Monroy-Hernández. 2023. Humans, AI, and Context: Understanding End-Users' Trust in a Real-World Computer Vision Application. In *Proceedings of the 2023 ACM Conference on Fairness, Accountability, and Transparency (FAccT '23)*. Association for Computing Machinery, New York, NY, USA, 77–88. <https://doi.org/10.1145/3593013.3593978>
- [33] John D. Lee and Katrina A. See. 2004. Trust in Automation: Designing for Appropriate Reliance. *Human Factors* 46, 1 (March 2004), 50–80. <https://doi.org/10.1518/hfes.46.1.50.30392> Publisher: SAGE Publications Inc.
- [34] Mina Lee, Percy Liang, and Qian Yang. 2022. CoAuthor: Designing a Human-AI Collaborative Writing Dataset for Exploring Language Model Capabilities. In *Proceedings of the 2022 CHI Conference on Human Factors in Computing Systems (CHI '22)*. Association for Computing Machinery, New York, NY, USA, 1–19. <https://doi.org/10.1145/3491102.3502030>
- [35] Jenny T Liang, Chenyang Yang, and Brad A Myers. 2024. A large-scale survey on the usability of ai programming assistants: Successes and challenges. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*. 1–13.
- [36] QVera Liao and S. Shyam Sundar. 2022. Designing for Responsible Trust in AI Systems: A Communication Perspective. In *2022 ACM Conference on Fairness, Accountability, and Transparency (FAccT '22)*. Association for Computing Machinery, New York, NY, USA, 1257–1268. <https://doi.org/10.1145/3531146.3533182>
- [37] Q. Vera Liao, Daniel Gruen, and Sarah Miller. 2020. Questioning the AI: Informing Design Practices for Explainable AI User Experiences. In *Proceedings of the 2020 CHI Conference on Human Factors in Computing Systems*. 1–15. <https://doi.org/10.1145/3313831.3376590> arXiv:2001.02478 [cs].
- [38] S. Lipner. 2004. The trustworthy computing security development lifecycle. In *20th Annual Computer Security Applications Conference*. 2–13. <https://doi.org/10.1109/CSAC.2004.41>
- [39] Ryan Louie, Andy Coenen, Cheng Zhi Huang, Michael Terry, and Carrie J. Cai. 2020. Novice-AI Music Co-Creation via AI-Steering Tools for Deep Generative Models. In *Proceedings of the 2020 CHI Conference on Human Factors in Computing Systems (CHI '20)*. Association for Computing Machinery, New York, NY, USA, 1–13. <https://doi.org/10.1145/3313831.3376739>
- [40] Xiao Ma, Swaroop Mishra, Ariel Liu, Sophie Su, Jilin Chen, Chinmay Kulkarni, Heng-Tze Cheng, Quoc Le, and Ed Chi. 2023. Beyond ChatBots: ExploreLLM for Structured Thoughts and Personalized Model Responses. *arXiv preprint arXiv:2312.00763* (2023).
- [41] Roger C. Mayer, James H. Davis, and F. David Schoorman. 1995. An Integrative Model Of Organizational Trust. *Academy of Management Review* 20, 3 (July 1995), 709–734. <https://doi.org/10.5465/amr.1995.9508080335> Publisher: Academy of Management.
- [42] Siddharth Mehrotra, Carolina Centeio Jorge, Catholijn M. Jonker, and Myrthe L. Tielman. 2023. Integrity Based Explanations for Fostering Appropriate Trust in AI Agents. *ACM Transactions on Interactive Intelligent Systems* (July 2023). <https://doi.org/10.1145/3610578> Just Accepted.
- [43] Swati Mishra and Jeffrey M. Rzeszutarski. 2021. Crowdsourcing and Evaluating Concept-driven Explanations of Machine Learning Models. *Proceedings of the ACM on Human-Computer Interaction* 5, CSCW1 (April 2021), 139:1–139:26. <https://doi.org/10.1145/3449213>
- [44] Hussein Mozannar, Gagan Bansal, Adam Fourney, and Eric Horvitz. 2023. Reading Between the Lines: Modeling User Behavior and Costs in AI-Assisted Programming. <https://doi.org/10.48550/arXiv.2210.14306> arXiv:2210.14306 [cs].
- [45] Robert Münscher and Torsten M. Kühlmann. 2011. Using critical incident technique in trust research. *Handbook of research methods on trust* (2011), 161.
- [46] Emerson Murphy-Hill, Ciera Jaspas, Caitlin Sadowski, David Shepherd, Michael Phillips, Collin Winter, Andrea Knight, Edward Smith, and Matthew Jorde. 2021. What Predicts Software Developers' Productivity? *IEEE Transactions on Software Engineering* 47, 3 (2021), 582–594. <https://doi.org/10.1109/TSE.2019.2900308>
- [47] Annette M. O'Connor, Guy Tsafnat, James Thomas, Paul Glasziou, Stephen B. Gilbert, and Brian Hutton. 2019. A question of trust: can we build an evidence base to gain trust in systematic review automation technologies? *Systematic Reviews* 8, 1 (June 2019), 143. <https://doi.org/10.1186/s13643-019-1062-0>
- [48] Hammond Pearce, Baleegh Ahmad, Benjamin Tan, Brendan Dolan-Gavitt, and Ramesh Karri. 2022. Asleep at the Keyboard? Assessing the Security of GitHub Copilot's Code Contributions. In *2022 IEEE Symposium on Security and Privacy (SP)*. 754–768. <https://doi.org/10.1109/SP46214.2022.9833571> ISSN: 2375-1207.
- [49] Neil Perry, Megha Srivastava, Deepak Kumar, and Dan Boneh. 2022. Do Users Write More Insecure Code with AI Assistants? <https://doi.org/10.48550/arXiv.2211.03622> arXiv:2211.03622 [cs].
- [50] Advait Sarkar, Andrew D. Gordon, Carina Negreanu, Christian Poelitz, Sruti Srinivasa Ragavan, and Ben Zorn. 2022. What is it like to program with artificial intelligence? <https://doi.org/10.48550/arXiv.2208.06213> arXiv:2208.06213 [cs].
- [51] Xinyue Shen, Zeyuan Chen, Michael Backes, and Yang Zhang. 2023. In ChatGPT We Trust? Measuring and Characterizing the Reliability of ChatGPT. <https://arxiv.org/abs/2304.08979v1>
- [52] Dominik Sobania, Dirk Schweim, and Franz Rothlauf. 2022. A comprehensive survey on program synthesis with evolutionary algorithms. *IEEE Transactions on Evolutionary Computation* (2022).
- [53] Jiao Sun, Q. Vera Liao, Michael Muller, Mayank Agarwal, Stephanie Houde, Kartik Talamadupula, and Justin D. Weisz. 2022. Investigating Explainability of Generative AI for Code through Scenario-based Design. In *27th International Conference on Intelligent User Interfaces (IUI '22)*. Association for Computing Machinery, New York, NY, USA, 212–228. <https://doi.org/10.1145/3490099.3511119>
- [54] Suzanne Tolmeijer, Markus Christen, Serhiy Kandul, Markus Kneer, and Abraham Bernstein. 2022. Capable but Amoral? Comparing AI and Human Expert Collaboration in Ethical Decision Making. In *Proceedings of the 2022 CHI Conference on Human Factors in Computing Systems (CHI '22)*. Association for Computing Machinery, New York, NY, USA, 1–17. <https://doi.org/10.1145/3491102.3517732>
- [55] Priyan Vaithilingam, Tianyi Zhang, and Elena L. Glassman. 2022. Expectation vs. Experience: Evaluating the Usability of Code Generation Tools Powered by Large Language Models. In *Extended Abstracts of the 2022 CHI Conference on*

- Human Factors in Computing Systems (CHI EA '22)*. Association for Computing Machinery, New York, NY, USA, 1–7. <https://doi.org/10.1145/3491101.3519665>
- [56] Oleksandra Vereschak, Gilles Bailly, and Baptiste Caramiaux. 2021. How to Evaluate Trust in AI-Assisted Decision Making? A Survey of Empirical Methodologies. *Proceedings of the ACM on Human-Computer Interaction* 5, CSCW2 (Oct. 2021), 327:1–327:39. <https://doi.org/10.1145/3476068>
- [57] Bryan Wang, Gang Li, and Yang Li. 2023. Enabling conversational interaction with mobile ui using large language models. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*. 1–17.
- [58] Justin D. Weisz, Michael Muller, Stephanie Houde, John Richards, Steven I. Ross, Fernando Martinez, Mayank Agarwal, and Kartik Talamadupula. 2021. Perfection Not Required? Human-AI Partnerships in Code Translation. In *26th International Conference on Intelligent User Interfaces (IUI '21)*. Association for Computing Machinery, New York, NY, USA, 402–412. <https://doi.org/10.1145/3397481.3450656>
- [59] Justin D. Weisz, Michael Muller, Steven I. Ross, Fernando Martinez, Stephanie Houde, Mayank Agarwal, Kartik Talamadupula, and John T. Richards. 2022. Better Together? An Evaluation of AI-Supported Code Translation. In *27th International Conference on Intelligent User Interfaces (IUI '22)*. Association for Computing Machinery, New York, NY, USA, 369–391. <https://doi.org/10.1145/3490099.3511157>
- [60] Eva Wendt, Bengt Fridlund, and Evy Lidell. 2004. Trust and Confirmation in a Gynecologic Examination Situation: A Critical Incident Technique Analysis. *Acta Obstetrica et Gynecologica Scandinavica* 83, 12 (2004), 1208–1215. <https://doi.org/10.1111/j.0001-6349.2004.00597.x>
- [61] David Gray Widder, Laura Dabbish, James D. Herbsleb, Alexandra Holloway, and Scott Davidoff. 2021. Trust in Collaborative Automation in High Stakes Software Engineering Work: A Case Study at NASA. In *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems (CHI '21)*. Association for Computing Machinery, New York, NY, USA, 1–13. <https://doi.org/10.1145/3411764.3445650>
- [62] Jim Witschey, Olga Zielinska, Allaire Welk, Emerson Murphy-Hill, Chris Mayhorn, and Thomas Zimmermann. 2015. Quantifying Developers' Adoption of Security Tools. In *Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering (Bergamo, Italy) (ESEC/FSE 2015)*. Association for Computing Machinery, New York, NY, USA, 260–271. <https://doi.org/10.1145/2786805.2786816>
- [63] Shundan Xiao, Jim Witschey, and Emerson Murphy-Hill. 2014. Social Influences on Secure Development Tool Adoption: Why Security Tools Spread. In *Proceedings of the 17th ACM Conference on Computer Supported Cooperative Work & Social Computing (Baltimore, Maryland, USA) (CSCW '14)*. Association for Computing Machinery, New York, NY, USA, 1095–1106. <https://doi.org/10.1145/2531602.2531722>
- [64] Frank F. Xu, Bogdan Vasilescu, and Graham Neubig. 2022. In-IDE Code Generation from Natural Language: Promise and Challenges. *ACM Transactions on Software Engineering and Methodology* 31, 2 (March 2022), 29:1–29:47. <https://doi.org/10.1145/3487569>
- [65] Fumeng Yang, Zhuanyi Huang, Jean Scholtz, and Dustin L. Arendt. 2020. How do visual explanations foster end users' appropriate trust in machine learning?. In *Proceedings of the 25th International Conference on Intelligent User Interfaces (IUI '20)*. Association for Computing Machinery, New York, NY, USA, 189–201. <https://doi.org/10.1145/3377325.3377480>
- [66] Qian Yang, Yuexing Hao, Kexin Quan, Stephen Yang, Yiran Zhao, Volodymyr Kuleshov, and Fei Wang. 2023. Harnessing Biomedical Literature to Calibrate Clinicians' Trust in AI Decision Support Systems. <https://doi.org/10.1145/3544548.3581393>
- [67] Rodrigo Yañez-Gallardo and Sandra Valenzuela-Suazo. 2012. Critical incidents of trust erosion in leadership of head nurses. *Revista Latino-Americana de Enfermagem* 20 (Feb. 2012), 143–150. <https://doi.org/10.1590/S0104-11692012000100019> Publisher: Escola de Enfermagem de Ribeirão Preto / Universidade de São Paulo.
- [68] JD Zamfirescu-Pereira, Richmond Y Wong, Bjoern Hartmann, and Qian Yang. 2023. Why Johnny can't prompt: how non-AI experts try (and fail) to design LLM prompts. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*. 1–21.
- [69] Yunfeng Zhang, Q. Vera Liao, and Rachel K. E. Bellamy. 2020. Effect of confidence and explanation on accuracy and trust calibration in AI-assisted decision making. In *Proceedings of the 2020 Conference on Fairness, Accountability, and Transparency (FAT* '20)*. Association for Computing Machinery, New York, NY, USA, 295–305. <https://doi.org/10.1145/3351095.3372852>
- [70] Yixuan Zhang, Nurul Suhaimi, Nutchanon Yongsatanchot, Joseph D Gaggiano, Miso Kim, Shivani A Patel, Yifan Sun, Stacy Marsella, Jacqueline Griffin, and Andrea G Parker. 2022. Shifting Trust: Examining How Trust and Distrust Emerge, Transform, and Collapse in COVID-19 Information Seeking. In *Proceedings of the 2022 CHI Conference on Human Factors in Computing Systems (CHI '22)*. Association for Computing Machinery, New York, NY, USA, 1–21. <https://doi.org/10.1145/3491102.3501889>
- [71] Albert Ziegler, Eirini Kalliamvakou, X. Alice Li, Andrew Rice, Devon Rifkin, Shawn Simister, Ganesh Sittampalam, and Edward Aftandilian. 2022. Productivity assessment of neural code completion. In *Proceedings of the 6th ACM SIGPLAN*

International Symposium on Machine Programming (MAPS 2022). Association for Computing Machinery, New York, NY, USA, 21–29. <https://doi.org/10.1145/3520312.3534864>

Table 1: The participants of Study 1. The Column *Exp* indicates the years of programming experience. The *Job Title* was self reported by the participants.

ID	Tool (Frequency)	Gender	Race	Age	Education	Job Title	Exp
P1	GitHub Copilot (Daily)	Male	White	25-34	Bachelor degree	Researcher	7
P2	GitHub Copilot (Weekly)	Male	Asian	18-24	Bachelor degree	Program Maleager	5
P3	GitHub Copilot (Monthly)	Male	White	45-54	Bachelor degree	Software Engineer	25
P4	GitHub Copilot (Monthly), Tabnine (Yearly)	Male	White	25-34	Bachelor degree	Software Engineer	12
P5	GitHub Copilot (Daily), Tabnine (Daily)	Male	Asian Indian	25-34	Ongoing Masters degree	Software engineer	9
P6	GitHub Copilot (Daily)	Female	White	25-34	Bachelor degree	Software Engineer	4
P7	GitHub Copilot (Monthly)	Male	White	25-34	PhD degree	Software Engineer	22
P8	GitHub Copilot (Weekly)	Male	Middle Eastern	35-44	Bachelor degree	Security Engineer	20
P9	GitHub Copilot (Weekly)	Male	Asian	25-34	Master degree	Software Engineer	8
P10	GitHub Copilot (Daily)	Male	Black or African American	25-34	High school diploma	Software Engineer	8
P11	GitHub Copilot (Daily)	Male	White	25-34	Bachelor degree	Software Engineer	9
P12	GitHub Copilot (Daily)	Male	White	35-44	Master degree	Software Engineer	21
P13	GitHub Copilot (Daily)	Male	White	18-24	Bachelor degree	Software engineer	6
P14	GitHub Copilot (Never)	Female	Asian	25-34	Bachelor degree	Software engineer	8
P15	GitHub Copilot (Daily)	Male	Hispanic or Latino	18-24	High school diploma	Software engineer intern	3
P16	GitHub Copilot (Daily)	Male	White	18-24	High school diploma	Software engineer intern	2
P17	GitHub Copilot (Never)	Male	Asian	25-34	Bachelor degree	Software engineer	5

Table 2: The participants of Study 2. The Column *Exp* indicates the years of programming experience.

ID	GitHub Copilot (Frequency)	Gender	Race	Age	Education	Exp
P1	I use the tool regularly	Female	White	35-44	PhD degree	4
P2	I use the tool regularly	Male	White	55-64	Bachelor degree	45
P3	I've tried the tool but no longer using it	Male	Asian	35-44	Bachelor degree	5
P4	I use the tool regularly	Male	Black or African American	25-34	Bachelor degree	6
P5	I use the tool regularly	Male	White	25-34	Bachelor degree	15
P6	I recently started using the tool	Male	White	25-34	Master degree	7
P7	I recently started using the tool	Female	Asian	25-34	Master degree	6
P8	I use the tool regularly	Female	Asian	18-24	Bachelor degree	3
P9	I use the tool regularly	Male	Asian	25-34	Master degree	6
P10	I've tried the tool but no longer using it	Male	Middle Eastern	18-24	High school diploma	6
P11	I use the tool regularly	Male	Black or African American	35-44	Master degree	11
P12	I use the tool regularly	Male	Asian	25-34	Master degree	7

A PARTICIPANT INFORMATION

B STUDY MATERIAL FOR STUDY 1

B.1 Example interview questions

The retrospective interviews were semi-structured, so the questions below only represent a general structure of the interviews. In the actual interviews, we followed up with participants whenever they mentioned topics relevant to their understanding of trust and the challenges they have in building appropriate trust.

- Could you tell us a bit more about the kind of programming project or tasks that you work on? What kind of development activities (e.g. front-end) are you typically involved in?
- What experience do you have with AI-powered code generation tools?
- How do you trust the AI tool?
- Can you walk me through the significant moments you collected?
 - What was your task?
 - How did you interact with the tool? [Feel free to share screen]
 - How do these interactions affect your trust in the tool? Why?

- Now think about your general experience interacting with the tool. How would you define trust?
- Where do you think the trust come from?
- What tasks do you trust/distrust the tool to do? Why?
- Were there moments where you trusted the AI tool but later realized that you shouldn't?
- Were there moments where you didn't trust the AI tool but later realized that you should?
- How has your perception of trust in the tool changed over time?
- How would you want to improve the design of AI-powered code generation tool so that you can trust it more appropriately?

B.2 Message sent to participants to collect significant moments

Hi [Participant Name], Thank you for signing up for the experience in AI-powered code generation tools research study. We would like to invite you for an interview to learn more about your experience. To prepare for the interview, we would like to invite you to collect significant

moments in your experience using AI-powered code generation tools (e.g., Copilot) in the next few days. Our goal is for you to collect these significant moments, so that you can reflect on your experience more concretely in the interview. Specifically, please aim to share 1 to 3 significant moments each day. Some examples of significant moments are when you are appreciative of, frustrated by, or hesitant/uncertain to use the AI-powered code generation tool (e.g., copilot). For each time you share, you can use one or two sentences to describe the instance, take a screenshot or share a snippet of code. You can share these in our chat directly. We will also send you a quick reminder message everyday morning during the week. In the case that you do not use AI-powered code generation tools during the day, it would also be helpful to share a quick update in the chat (e.g., did not use AI tools today). We will schedule an interview with you after you successfully complete the preparation phase (collect several significant moments).

C STUDY MATERIAL FOR STUDY 2

C.1 Example design probe questions

We begin the interview by briefing participants that:

- We are evaluating the prototype, not you, so feel free to comment on anything.
- Do not worry about the technical implementation of the designs. The purpose of the session is to get feedback on the concept of designs, instead of the feasibility of the designs.
- Do not worry about usability (e.g., layout, color, style) of the design
- Feel free to think aloud as you look at the design prototypes
- The code snippets are only placeholders. Try to imagine how you will use the design in your daily workflow.

Next, we ask the following questions to understand participants' understanding of trust.

- What experience do you have with AI-powered code generation tools, such as copilot?
- How do you trust the AI tool?
- How do you define trust?
- Are there challenges in knowing what to expect from the AI tool?
- Are there challenges in integrating the AI tool into your workflow?
- How would you want to design the interaction with the AI tool differently so that you can better judge when to trust the AI tool or not?

Present and give brief explanations of the mockups to participants one by one. For each mockup, ask the following questions:

- What do you think of this design?
- How might you use this feature in your daily coding task?
- Thinking about your overall experience interacting with the tool, to what extent do you think it will help you better judge when to trust copilot or not?
- Which of all the mockups is the most helpful in helping you judge when to trust copilot or not?

- What other features do you like to add to this prototype?
- What other features do you like to remove or change to this prototype?

D CODE BOOK FOR STUDY 1

Table 3 shows the codebook for the inductive thematic analysis in Study 1.

E CODE BOOK FOR STUDY 2

Table 4 shows the codebook for the thematic analysis in Study 2.

F FEATURES OF GITHUB COPILOT AND TABNINE AS OF JULY 2022

As of July 2022, GitHub Copilot was an AI-powered code generation tool that is integrated into code editors as shown in Figure 4. Based on the official website image of July 2022¹, GitHub Copilot “uses the OpenAI Codex to suggest code and entire functions in real-time, right from your editor.” It can generate whole lines or blocks of code based on the comments and preceding code snippets. Copilot also supports multiple programming languages and frameworks, including Python, JavaScript, etc. However, users cannot chat with the tool. Moreover, GitHub Copilot Chat, which allows users to interact with GitHub Copilot to ask and receive answers to coding-related questions, was not available. Features allowing users to select a snippet of code and ask natural language questions² also became available after our study. Similarly, Tabnine also only supported code completion within editors in July 2022³ and only supported a chat interface after our study.

G DESIGN CONCEPTS SHOWN IN THE STUDY

In Figure 5, 6, 7, we show the design prototypes that we showed to the study participants.

¹<https://web.archive.org/web/20220701014741/https://github.com/features/copilot> – Retrieval date: 05/02/2024

²<https://code.visualstudio.com/docs/copilot/overview> – Retrieval date: 05/02/2024

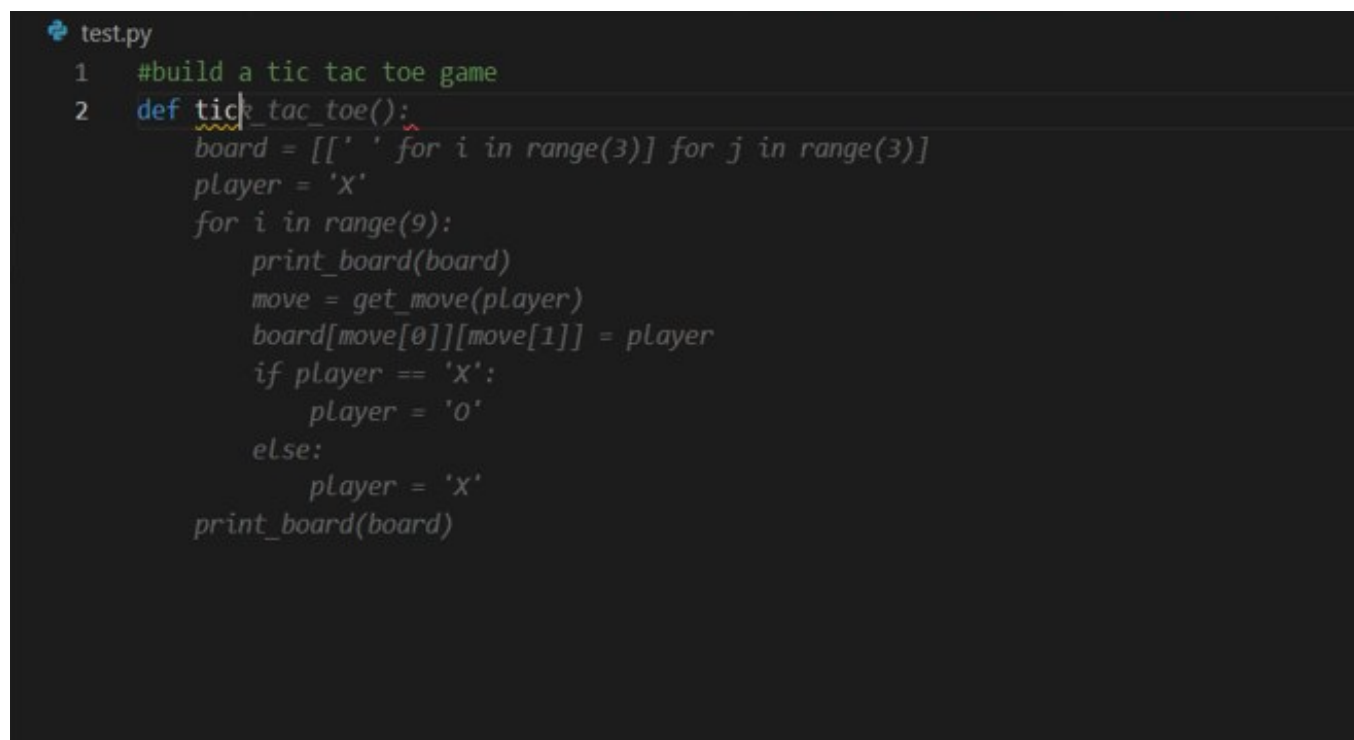
³<https://web.archive.org/web/20220705023816/https://www.tabnine.com/> – Retrieval date: 05/02/2024

Category	Code
How developers use AI tools	understand AI tool's utility over time
	assign different roles of AI tools
	expect different scope of suggestions
	validate before accepting suggestions
	willing to accept without validation
Factors affecting trust in AI tools	general trust perceptions
	compare AI tools to human
	effect on productivity
	stability of performance
	(mis)aligned expectation on AI tools
	ability to convey intention
	reliability of suggestions
	concerns around privacy and security
	transparency of model mechanism
	trust varied by complexity of task
	trust varied by the granularity of expected suggestion
	trust varied by programming language
Evaluating specific AI suggestions	trust varied by stake of task
	trust varied by individual factors
	local judgement differ from global trust perception
	global trust affects local judgement
Challenges in building trust in AI tools	knowing the exact context help evaluate AI suggestions
	explanation help evaluate AI suggestions
	lack of support in onboarding experience
	trust perception shift over time
	initial expectation affects trust building
	build trust via intentional experimentation
	prior knowledge shapes trust perception
	success and failure cases shape trust
	trial and error to build trust in AI
	want to understand the limits of AI tools
	evaluate suggestion based on external references
	fixing AI's error affects trust
	challenges in validation
	learning how to control AI tools
	assign too much responsibility on AI tools
	integrate AI tools in workflow
	expect the AI tools' performance to grow over time
	develop folk theory of how AI works

Table 3: Codebook for inductive thematic analysis in Study 1

Design concept	Code
Usage statistics dashboard	demonstrate values of AI tools
	support exploration of AI capabilities
	help understand the limitation of AI tools
	privacy concern around behavior analytics
	high effort to interpret the stats
Quality indicators	help guide decisions on whether to accept suggestion or not
	help show vulnerability and demystifying AI tools
	helpful signals to make trust judgement
	difficult to interpret numbers
	potential to introduce bias
Control mechanisms	help set boundaries and align intentions
	help build expectations on AI tools
	settings are hard to understand
	additional effort of using AI tools

Table 4: Codebook for thematic analysis in Study 2

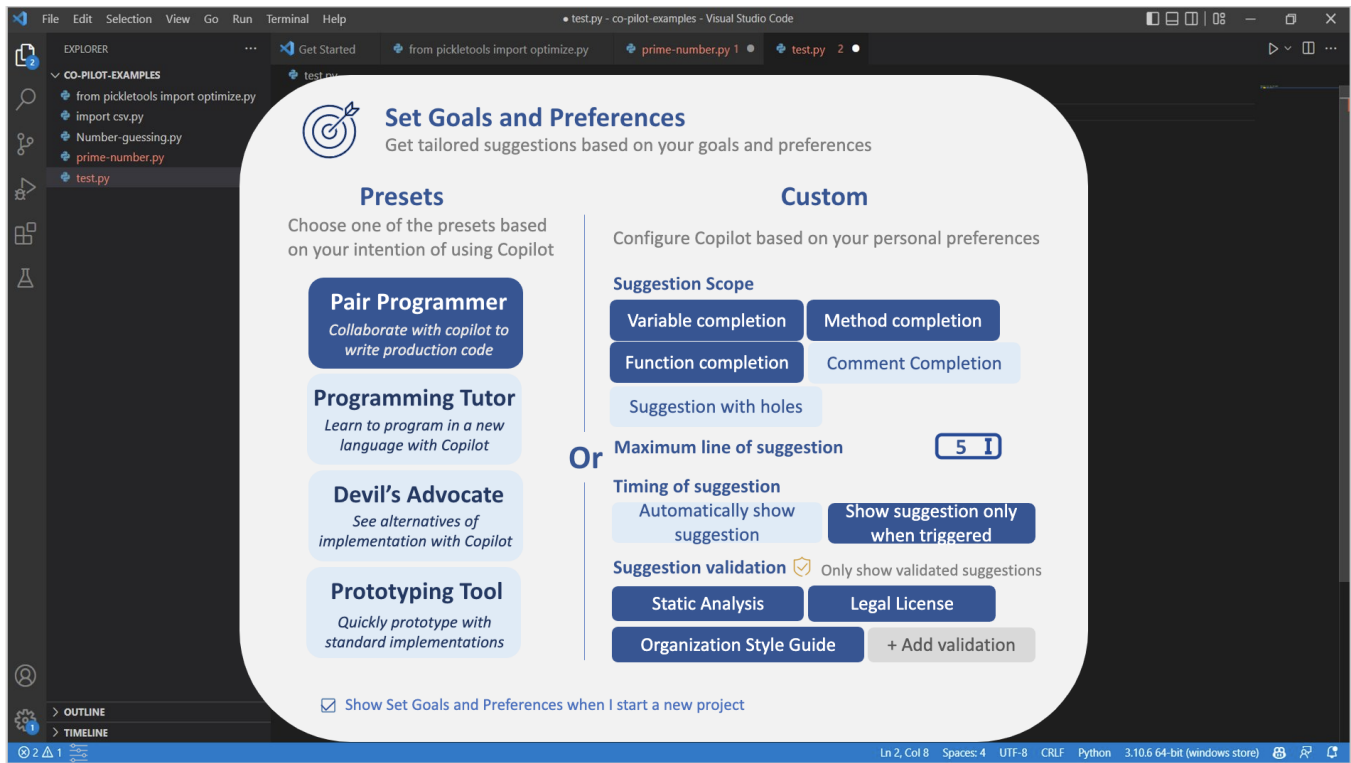


```

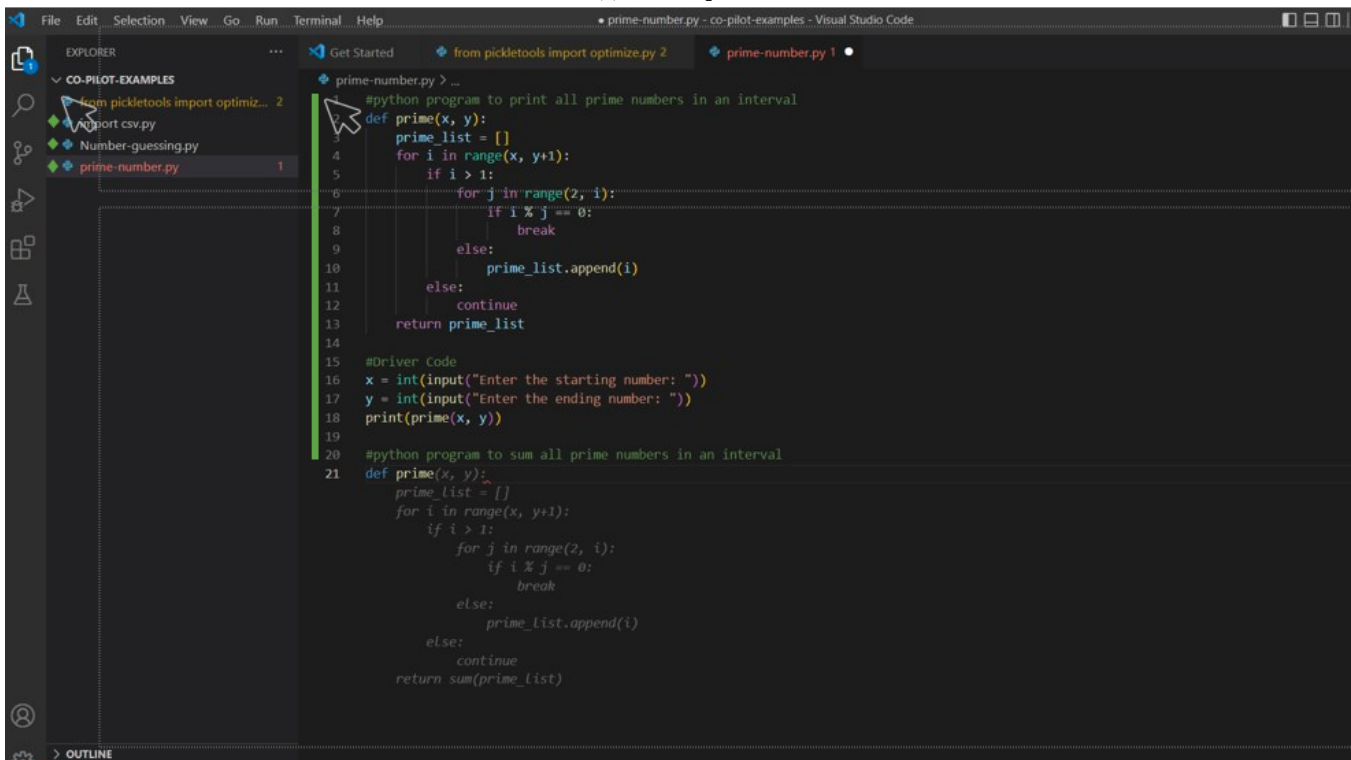
test.py
1  #build a tic tac toe game
2  def tic_tac_toe():
    board = [[' ' for i in range(3)] for j in range(3)]
    player = 'X'
    for i in range(9):
        print_board(board)
        move = get_move(player)
        board[move[0]][move[1]] = player
        if player == 'X':
            player = 'O'
        else:
            player = 'X'
    print_board(board)

```

Figure 4: GitHub Copilot interface, as of July 2022

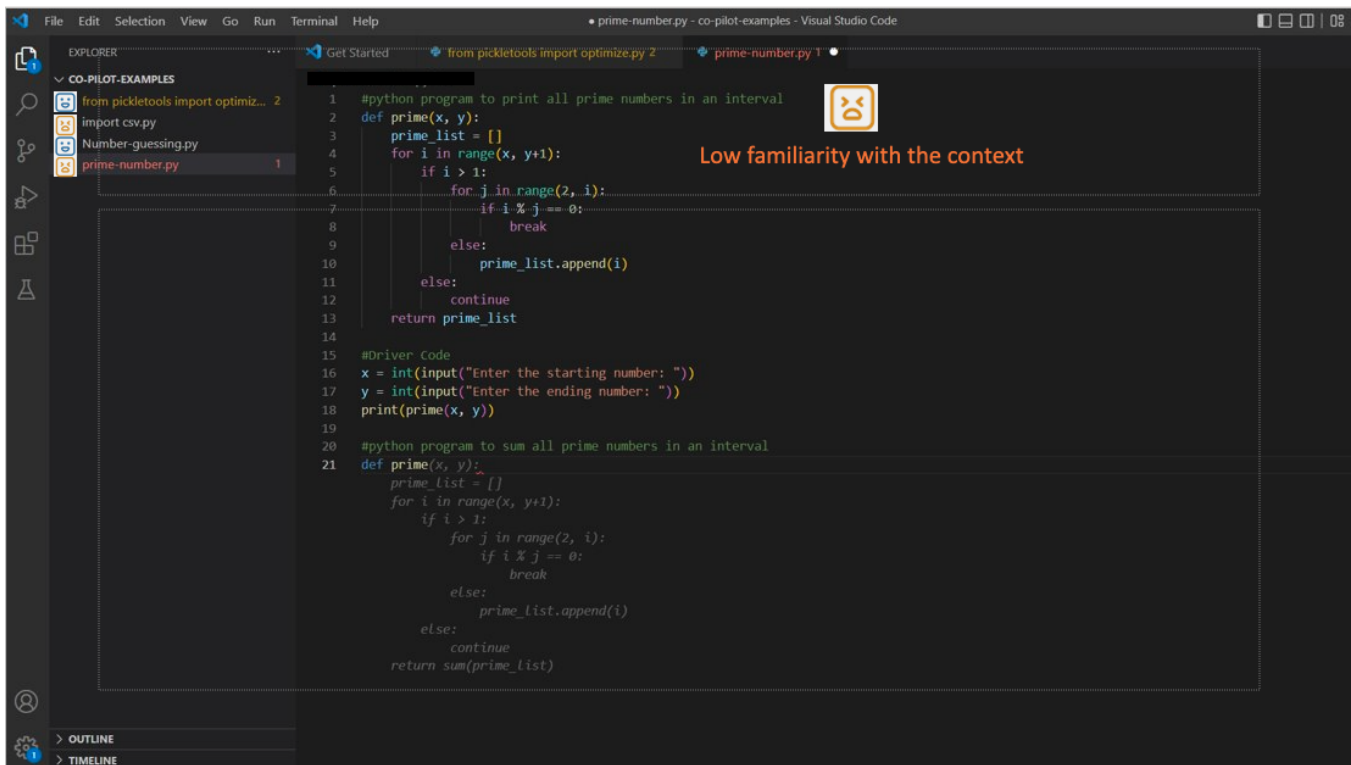


(a) Control panel

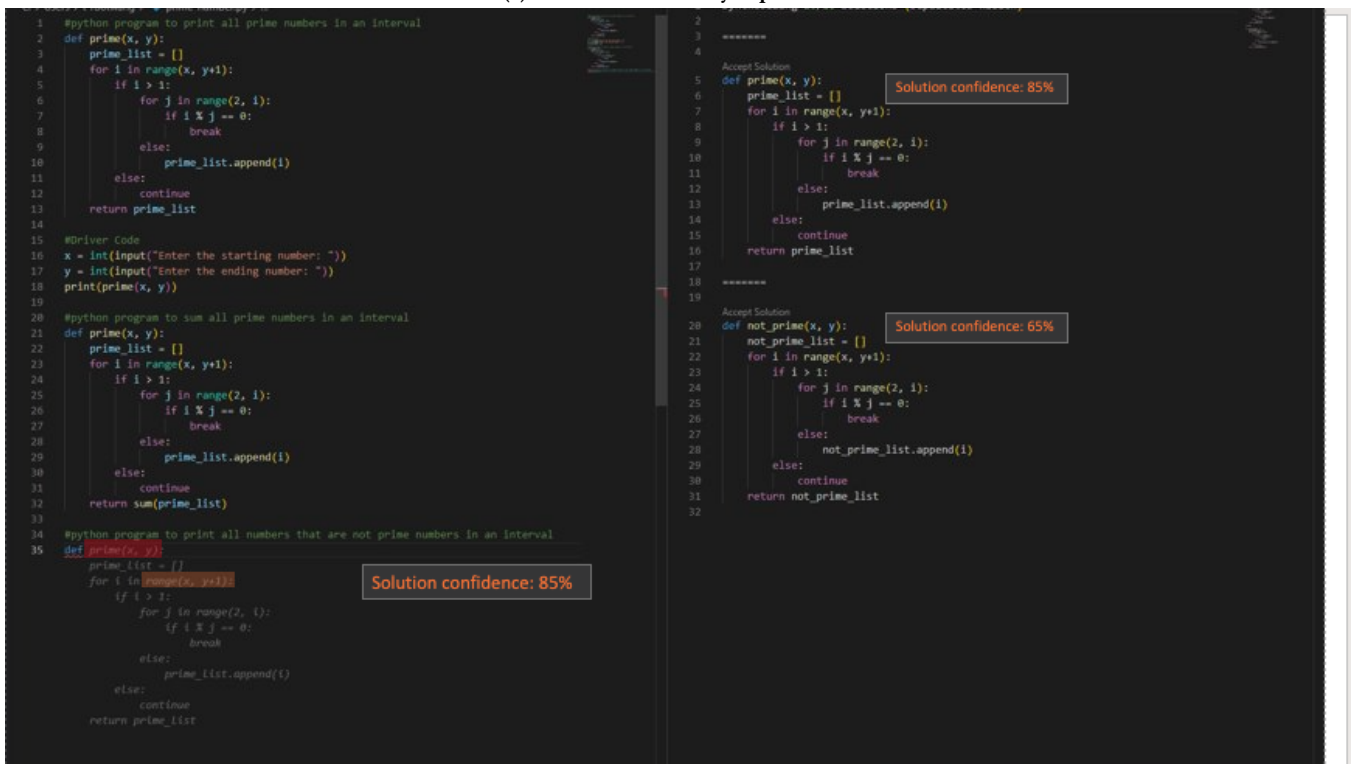


(b) Context slider

Figure 5: Group 1: Control mechanisms



(a) File-level familiarity explanation



(b) Solution-level and token-level confidence explanations

Figure 6: Group 2: Quality indicators of AI suggestions

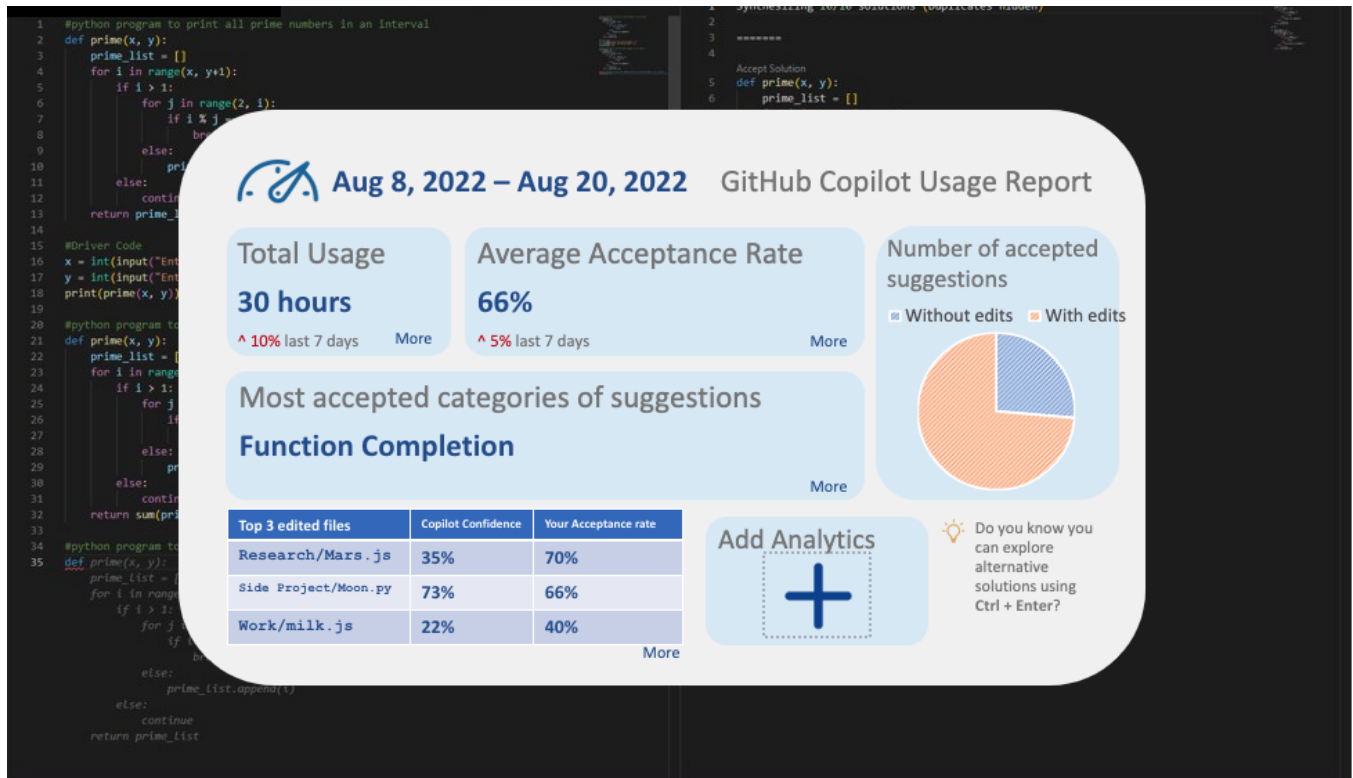


Figure 7: Group 3: Usage statistics dashboard